Dependency inversion is the process of taking a single direction class-to-class dependency and inserting an abstraction between them. The dependent class now points to the interface, while on the other side the target class is now pointing (with an implementation arrow) to the interface. To sum up, the DIP turns a design like this…

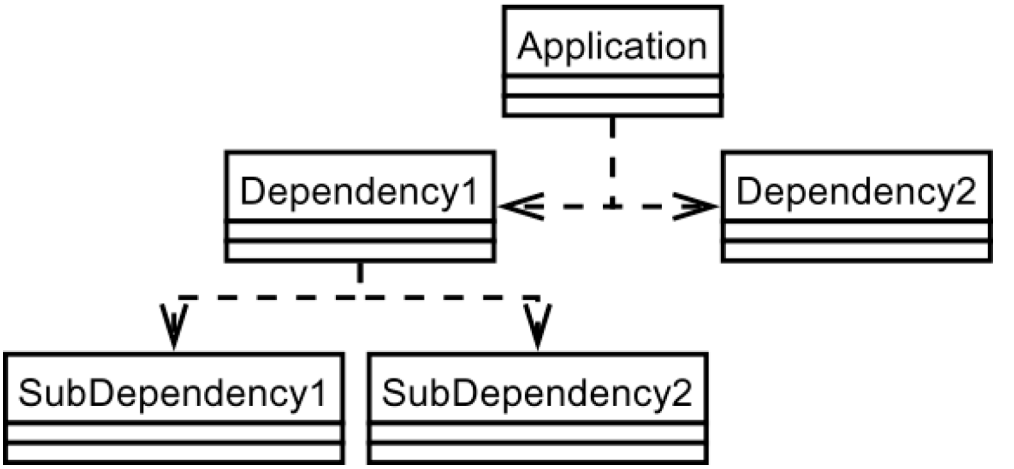



***Figure 10-16.*** *Each layer points toward a lower layer*

…into a model like this…

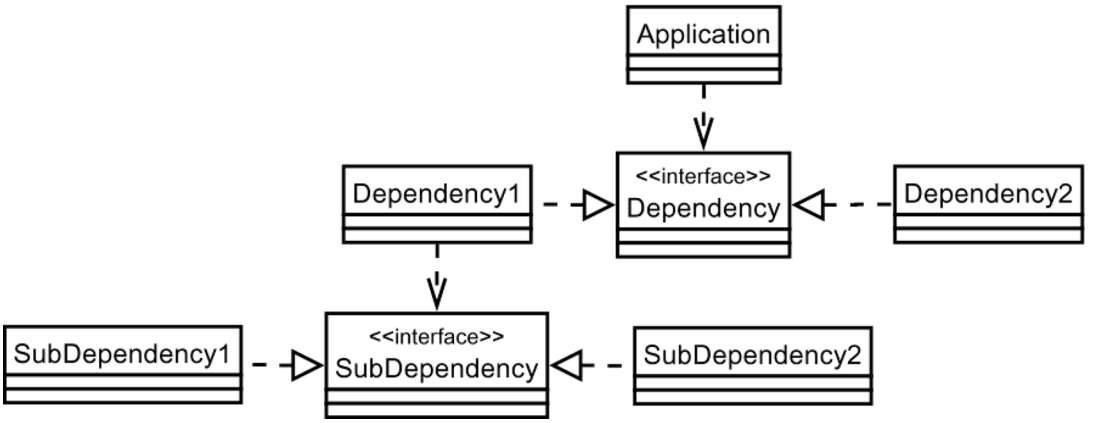



***Figure 10-17.*** *Each layer points toward an abstraction*

…and the result is code that is much easier to manage. Client classes use the abstraction (interface or abstract class) in their arguments; dependency classes implement the interface. We are now free to add as many new implementing classes as we like, and we don't have to modify the client class each time we do.